

# РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

## ОТЧЕТ

### ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

дисциплина: *Архитектура компьютера*

Студент: Кабанов Иван Олегович

Группа: НКАбд-03-25

МОСКВА

2025 г.

# Оглавление

|                                                            |    |
|------------------------------------------------------------|----|
| Цель работы _____                                          | 3  |
| 9.4. Выполнение лабораторной работы _____                  | 3  |
| 9.4.1. Реализация подпрограмм в NASM _____                 | 3  |
| 9.4.2. Отладка программ с помощью GDB _____                | 6  |
| 9.4.2.1. Добавление точек останова _____                   | 10 |
| 9.4.2.2. Работа с данными программы в GDB _____            | 10 |
| 9.4.2.3. Обработка аргументов командной строки в GDB _____ | 16 |
| 9.5. Задание для самостоятельной работы _____              | 18 |
| Вывод _____                                                | 24 |

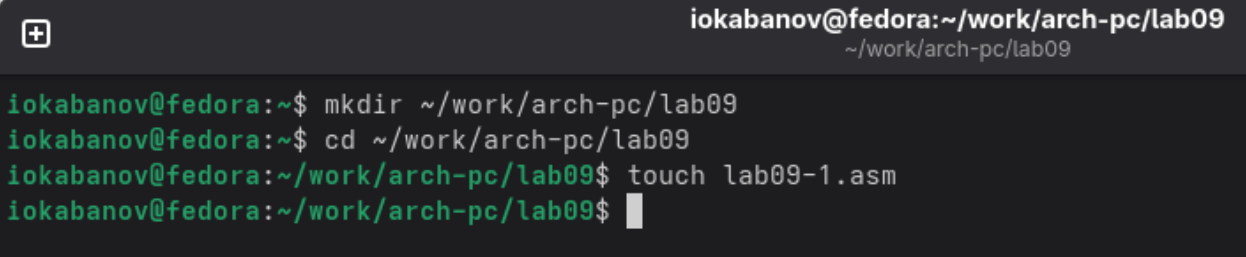
## Цель работы

Приобретение навыков написания программ с использованием подпрограмм.  
Знакомство с методами отладки при помощи GDB и его основными возможностями.

### 9.4. Выполнение лабораторной работы

#### 9.4.1. Реализация подпрограмм в NASM

Создаем каталог для программ лабораторной работы № 9, переходим в него и создаем файл lab09-1.asm. Вводим в файл lab09-1.asm текст программы из листинга 9.1, создаем исполняемый файл и проверяем его работу.

A terminal window with a dark background. The title bar shows a plus icon on the left and the text 'iokabanov@fedora:~/work/arch-pc/lab09' on the right. Below the title bar, the terminal shows four lines of commands and their outputs. The first line is 'mkdir ~/work/arch-pc/lab09' with output 'iokabanov@fedora:~\$ mkdir ~/work/arch-pc/lab09'. The second line is 'cd ~/work/arch-pc/lab09' with output 'iokabanov@fedora:~\$ cd ~/work/arch-pc/lab09'. The third line is 'touch lab09-1.asm' with output 'iokabanov@fedora:~/work/arch-pc/lab09\$ touch lab09-1.asm'. The fourth line shows the prompt 'iokabanov@fedora:~/work/arch-pc/lab09\$' followed by a cursor. The text is in a green monospace font.

```
iokabanov@fedora:~$ mkdir ~/work/arch-pc/lab09
iokabanov@fedora:~$ cd ~/work/arch-pc/lab09
iokabanov@fedora:~/work/arch-pc/lab09$ touch lab09-1.asm
iokabanov@fedora:~/work/arch-pc/lab09$
```

Рис. 1. Создаем lab09-1.asm

```

mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.pwd.EoLc5e
file://10.0.2.15/home/iokabanov/work/arch-pc/lab09
lab09-1.asm [----] 0 L: [ 1+ 0 1/ 48] *(0 / 760b) 0037 0x025
include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB '2x+7=',0

SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
GLOBAL _start
_start:

;-----
; Основная программа
;-----

mov eax, msg
call sprint

mov ecx, x
mov edx, 80
call sread

mov eax, x
call atoi

call _calcul ; Вызов подпрограммы _calcul

mov eax, result
call sprint
mov eax, [res]
call iprintLF

call quit

;-----
; Подпрограмма вычисления
; выражения "2x+7"
_calcul:

```

Рис. 2. Программа из листинга 9.1

```

mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.pwd.EoLc5e
file://10.0.2.15/home/iokabanov/work/arch-pc/lab09
lab09-1.asm [----] 0 L: [ 7+10 17/ 48] *(267 / 760b) 0059 0x03B
SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
GLOBAL _start
_start:

;-----
; Основная программа
;-----

mov eax, msg
call sprint

mov ecx, x
mov edx, 80
call sread

mov eax, x
call atoi

call _calcul ; Вызов подпрограммы _calcul

mov eax, result
call sprint
mov eax, [res]
call iprintLF

call quit

;-----
; Подпрограмма вычисления
; выражения "2x+7"
_calcul:
    mov ebx, 2
    mul ebx
    add eax, 7
    mov [res], eax
    ret ; выход из подпрограммы

```

Рис. 3. Программа из листинга 9.1 (продолжение)

```
iokabanov@fedora:~/work/arch-pc/lab09
~/work/arch-pc/lab09

iokabanov@fedora:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
iokabanov@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
iokabanov@fedora:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 3
2x+7=13
iokabanov@fedora:~/work/arch-pc/lab09$
```

Рис. 4. Запуск программы из листинга 9-1

Изменим текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения  $f(g(x))$ , где  $x$  вводится с клавиатуры,  $f(x) = 2x + 7$ ,  $g(x) = 3x - 1$ .

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.pwd.5su4lL
file:///10.0.2.15/home/iokabanov/work/arch-pc/lab09

lab09-1.asm  [----]  8 L: [ 1+41  42/ 61] *(668 /1118b) 0010 0x00A
#include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB 'f(g(x)) = 2(3x-1)+7 = ',0

SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
GLOBAL _start
_start:

;-----
; Основная программа
;-----

mov eax, msg
call sprint

mov ecx, x
mov edx, 80
call sread

mov eax, x
call atoi

call _calcul ; Вызов подпрограммы _calcul

mov eax, result
call sprint
mov eax, [res]
call iprintf

call quit

;-----
; Подпрограмма вычисления
; выражения f(g(x))
_calcul:
```

Рис. 5. Измененная программа из листинга 9.1

```

mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.pwd.5su4lL
file://10.0.2.15/home/iokabanov/work/arch-pc/lab09
lab09-1.asm [----] 0 L: [ 20+41 61/ 61] *(1118/1118b) <EOF>
call sprint

mov ecx, x
mov edx, 80
call sread

mov eax, x
call atoi

call _calcul ; Вызов подпрограммы _calcul

mov eax, result
call sprint
mov eax, [res]
call iprintLF

call quit

;-----
; Подпрограмма вычисления
; выражения f(g(x))
_calcul:
    call _subcalcul
    ....
    mov ebx, 2
    mul ebx
    add eax, 7
    mov [res], eax

    ret ; выход из подпрограммы
    ....
;-----
; Подпрограмма вычисления g(x) = 3x - 1
_subcalcul:
    mov ebx, 3
    mul ebx ; EAX = x * 3
    sub eax, 1 ; EAX = EAX - 1
    ....
    ret ; выход из подпрограммы _subcalcul, возврат в _calcul

```

Рис. 6. Измененная программа из листинга 9.1  
(продолжение)

```

iokabanov@fedora:~/work/arch-pc/lab09
~/work/arch-pc/lab09
iokabanov@fedora:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm -o lab09-1new.o
iokabanov@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1new lab09-1new.o
iokabanov@fedora:~/work/arch-pc/lab09$ ./lab09-1new
Введите x: 3
f(g(x)) = 2(3x-1)+7 = 23
iokabanov@fedora:~/work/arch-pc/lab09$

```

Рис. 7. Запуск измененной программы из листинга 9.1

### 9.4.2. Отладка программ с помощью GDB

Создаем файл lab09-2.asm с текстом программы из листинга 9.2, печатающей сообщение Hello world! Получаем исполняемый файл, проводя трансляцию с ключом ‘-g’. Проверяем работу программы, запустив ее в оболочке GDB с помощью команды run.

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.pwd.e3cUvm
file:///10.0.2.15/home/iokabanov/work/arch-pc/lab09

lab09-2.asm      [-M--]  8 L: [ 1+ 1  2/ 27] *(15 / 322b) 0109 0x06D
SECTION .data
<----->msg1: db "Hello, ",0x0
<----->msg1Len: equ $ - msg1
<----->
<----->msg2: db "world!",0xa
<----->msg2Len: equ $ - msg2
<----->
SECTION .text
<----->global _start
<----->
_start:
<----->mov eax, 4
<----->mov ebx, 1
<----->mov ecx, msg1
<----->mov edx, msg1Len
<----->int 0x80
<----->
<----->mov eax, 4
<----->mov ebx, 1
<----->mov ecx, msg2
<----->mov edx, msg2Len
<----->int 0x80
<----->
<----->mov eax, 1
<----->mov ebx, 0
<----->int 0x80
```

Рис. 8. Программа из листинга 9.2

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

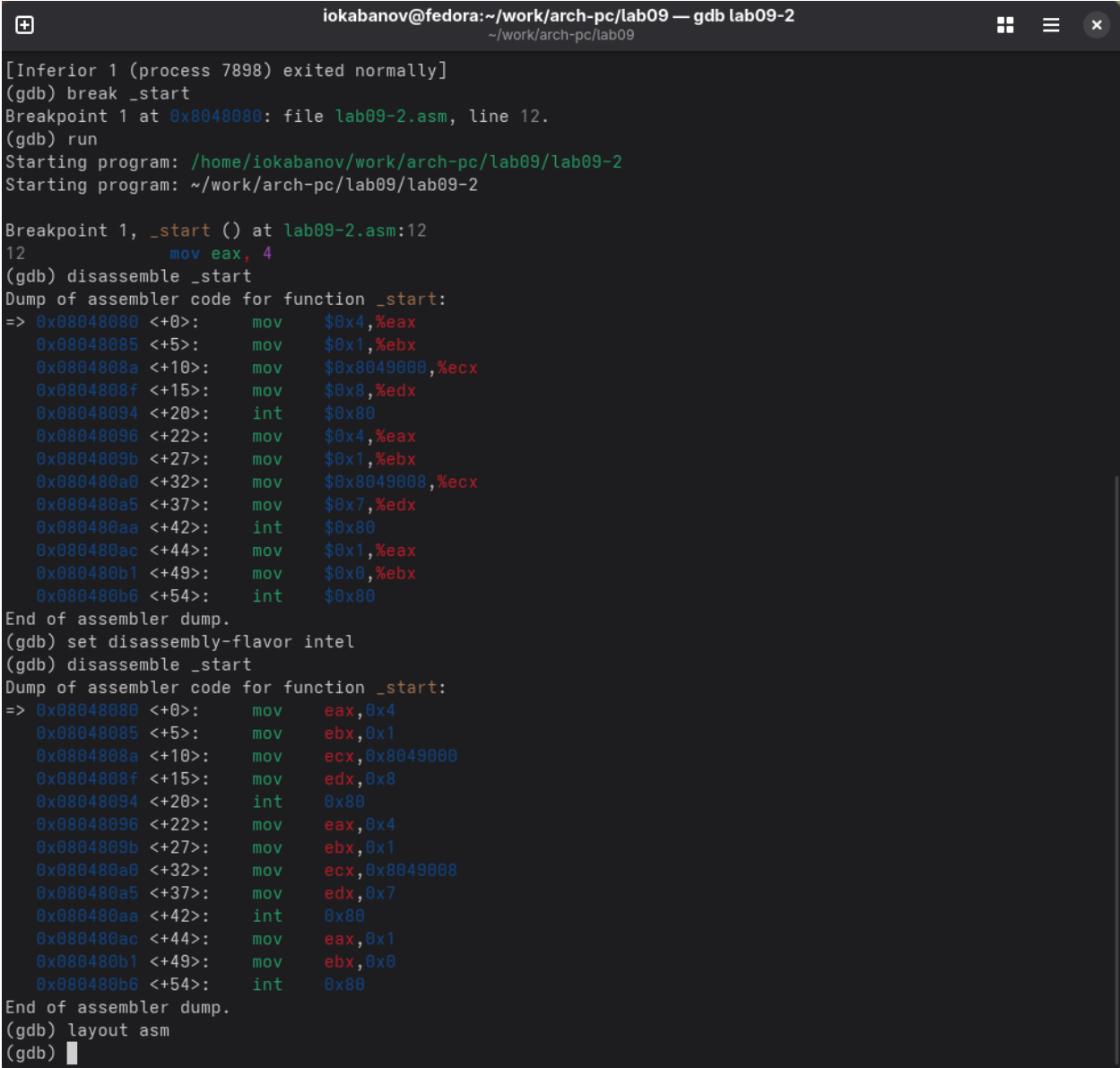
iokabanov@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
iokabanov@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
iokabanov@fedora:~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: ~/work/arch-pc/lab09/lab09-2
Starting program: /home/iokabanov/work/arch-pc/lab09/lab09-2
Starting program: ~/work/arch-pc/lab09/lab09-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading 51.96 K separate debug info for system-supplied DS0 at 0xf7ffc000
Hello, world!
```

Рис. 9. Запуск программы из листинга 9.2 в оболочке GDB

Для более подробного анализа программы устанавливаем брейкпоинт на метку `_start` и запускаем её. С помощью команды `disassemble` смотрим дисассимилированный код программы с метки `_start`. Введя команду `set disassembly_flavor intel` переключаемся на отображение команд с Intel'овским синтаксисом. С помощью `layout asm` и `layout regs` включаем режим псевдографики.



```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

[Inferior 1 (process 7898) exited normally]
(gdb) break _start
Breakpoint 1 at 0x08048080: file lab09-2.asm, line 12.
(gdb) run
Starting program: /home/iokabanov/work/arch-pc/lab09/lab09-2
Starting program: ~/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:12
12      mov     eax, 4
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:    mov     $0x4,%eax
0x08048085 <+5>:    mov     $0x1,%ebx
0x0804808a <+10>:   mov     $0x8049000,%ecx
0x0804808f <+15>:   mov     $0x8,%edx
0x08048094 <+20>:   int     $0x80
0x08048096 <+22>:   mov     $0x4,%eax
0x0804809b <+27>:   mov     $0x1,%ebx
0x080480a0 <+32>:   mov     $0x8049008,%ecx
0x080480a5 <+37>:   mov     $0x7,%edx
0x080480aa <+42>:   int     $0x80
0x080480ac <+44>:   mov     $0x1,%eax
0x080480b1 <+49>:   mov     $0x0,%ebx
0x080480b6 <+54>:   int     $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:    mov     eax,0x4
0x08048085 <+5>:    mov     ebx,0x1
0x0804808a <+10>:   mov     ecx,0x8049000
0x0804808f <+15>:   mov     edx,0x8
0x08048094 <+20>:   int     0x80
0x08048096 <+22>:   mov     eax,0x4
0x0804809b <+27>:   mov     ebx,0x1
0x080480a0 <+32>:   mov     ecx,0x8049008
0x080480a5 <+37>:   mov     edx,0x7
0x080480aa <+42>:   int     0x80
0x080480ac <+44>:   mov     eax,0x1
0x080480b1 <+49>:   mov     ebx,0x0
0x080480b6 <+54>:   int     0x80
End of assembler dump.
(gdb) layout asm
(gdb)
```

Рис. 10. Переключение на отображение команд с Intel'овским синтаксисом.



На рис. 10 можно увидеть следующие различия отображения синтаксиса машинных команд в режимах АТТ и Intel:

1. Порядок операндов:

АТ&Т: Источник → Назначение (`mov $0x4, %eax`).

Intel: Назначение ← Источник (`mov eax, 0x4`).

2. Написание регистров:

АТ&Т: Пишутся с символом % перед названием (`%eax`).

Intel: Пишутся чисто, без префиксов (`eax`).

3. Написание констант:

АТ&Т: Пишутся с символом \$ перед числом (`$0x80`).

Intel: Пишутся без префиксов (`0x80`).

Помимо различий на рис. 10 также стоит упомянуть:

4. Размер операции (суффиксы)

АТ&Т: Размер данных указывается суффиксом у команды (`b=byte, w=word, l=long/dword, q=quad`).

Пример: `movl, addb, subq`.

Intel: Команда обычно чистая, размер определяется операндами или явным указанием типа.

Пример: `mov, add, sub` (или `DWORD PTR`).

5. Адресация памяти (разыменование)

АТ&Т: Использует круглые скобки и специфический формат смещение(база, индекс, множитель).

Пример: `8(%ebp)` или `-4(%rdi, %rax, 8)`.

Intel: Использует квадратные скобки и математическую запись [база + индекс\*множитель + смещение].

Пример: `[ebp + 8]` или `[rdi + rax*8 - 4]`.

6. Дальние переходы (Far Jumps)

АТ&Т: Используется префикс `l`.

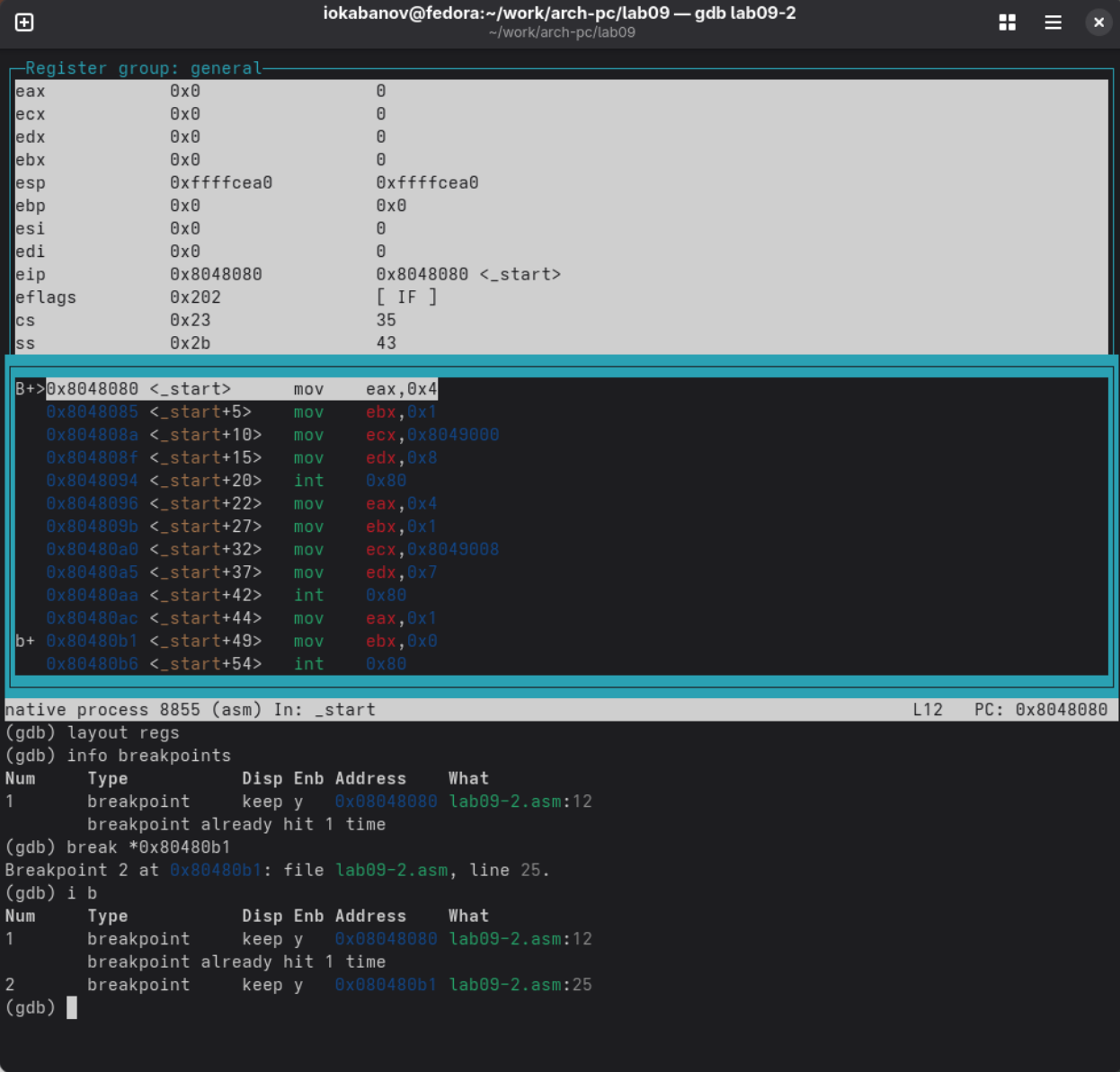
Пример: `ljmp $section, $offset`.

Intel: Используется ключевое слово `far`.

Пример: `jmp far section:offset`.

#### 9.4.2.1. Добавление точек останова

Проверяем установленную на предыдущих шагах точку останова с помощью `info breakpoints`. Определяем адрес предпоследней инструкции (`mov ebx,0x0`) и устанавливаем еще одну точку останова по адресу.



The screenshot shows the GDB interface with the following content:

**Register group: general**

| Register | Value      | Comment            |
|----------|------------|--------------------|
| eax      | 0x0        |                    |
| ecx      | 0x0        |                    |
| edx      | 0x0        |                    |
| ebx      | 0x0        |                    |
| esp      | 0xffffcea0 | 0xffffcea0         |
| ebp      | 0x0        | 0x0                |
| esi      | 0x0        | 0                  |
| edi      | 0x0        | 0                  |
| eip      | 0x8048080  | 0x8048080 <_start> |
| eflags   | 0x202      | [ IF ]             |
| cs       | 0x23       | 35                 |
| ss       | 0x2b       | 43                 |

**Disassembly:**

```
B+>0x8048080 <_start> mov    eax,0x4
0x8048085 <_start+5> mov    ebx,0x1
0x804808a <_start+10> mov    ecx,0x8049000
0x804808f <_start+15> mov    edx,0x8
0x8048094 <_start+20> int    0x80
0x8048096 <_start+22> mov    eax,0x4
0x804809b <_start+27> mov    ebx,0x1
0x80480a0 <_start+32> mov    ecx,0x8049008
0x80480a5 <_start+37> mov    edx,0x7
0x80480aa <_start+42> int    0x80
0x80480ac <_start+44> mov    eax,0x1
b+ 0x80480b1 <_start+49> mov    ebx,0x0
0x80480b6 <_start+54> int    0x80
```

**Breakpoints:**

```
native process 8855 (asm) In: _start L12 PC: 0x8048080
(gdb) layout regs
(gdb) info breakpoints
Num   Type             Disp Enb Address      What
1     breakpoint       keep y 0x08048080 lab09-2.asm:12
      breakpoint already hit 1 time
(gdb) break *0x80480b1
Breakpoint 2 at 0x80480b1: file lab09-2.asm, line 25.
(gdb) i b
Num   Type             Disp Enb Address      What
1     breakpoint       keep y 0x08048080 lab09-2.asm:12
      breakpoint already hit 1 time
2     breakpoint       keep y 0x080480b1 lab09-2.asm:25
(gdb)
```

Рис. 11. Точки останова

#### 9.4.2.2. Работа с данными программы в GDB

Выполняем 5 инструкций с помощью команды `si` и прослеживаем изменение значений регистров. Меняются значения регистров `ebx`, `ecx`, `edx`, `eax`, `eip`.

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/.work/arch-pc/lab09

Register group: general
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048085 0x8048085 <_start+5>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+ 0x8048080 <_start>      mov     eax,0x4
>0x8048085 <_start+5>      mov     ebx,0x1
0x804808a <_start+10>     mov     ecx,0x8049000
0x804808f <_start+15>     mov     edx,0x8
0x8048094 <_start+20>     int     0x80
0x8048096 <_start+22>     mov     eax,0x4
0x804809b <_start+27>     mov     ebx,0x1
0x80480a0 <_start+32>     mov     ecx,0x8049008
0x80480a5 <_start+37>     mov     edx,0x7
0x80480aa <_start+42>     int     0x80
0x80480ac <_start+44>     mov     eax,0x1
b+ 0x80480b1 <_start+49>     mov     ebx,0x0
0x80480b6 <_start+54>     int     0x80

native process 8855 (asm) In: _start L13 PC: 0x8048085
(gdb) layout regs
(gdb) info breakpoints
Num   Type      Disp Enb Address  What
1     breakpoint keep y  0x08048080 lab09-2.asm:12
      breakpoint already hit 1 time
(gdb) break *0x80480b1
Breakpoint 2 at 0x80480b1: file lab09-2.asm, line 25.
(gdb) i b
Num   Type      Disp Enb Address  What
1     breakpoint keep y  0x08048080 lab09-2.asm:12
      breakpoint already hit 1 time
2     breakpoint keep y  0x080480b1 lab09-2.asm:25
(gdb) si
(gdb) |
```

Рис. 12. Выполнение 1 инструкции с помощью si

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/.work/arch-pc/lab09

Register group: general
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x1      1
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804808a 0x804808a <_start+10>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+ 0x8048080 <_start>      mov     eax,0x4
0x8048085 <_start+5>      mov     ebx,0x1
>0x804808a <_start+10>     mov     ecx,0x8049000
0x804808f <_start+15>     mov     edx,0x8
0x8048094 <_start+20>     int     0x80
0x8048096 <_start+22>     mov     eax,0x4
0x804809b <_start+27>     mov     ebx,0x1
0x80480a0 <_start+32>     mov     ecx,0x8049008
0x80480a5 <_start+37>     mov     edx,0x7
0x80480aa <_start+42>     int     0x80
0x80480ac <_start+44>     mov     eax,0x1
b+ 0x80480b1 <_start+49>     mov     ebx,0x0
0x80480b6 <_start+54>     int     0x80

native process 8855 (asm) In: _start L14 PC: 0x804808a
(gdb) layout regs
(gdb) info breakpoints
Num   Type      Disp Enb Address  What
1     breakpoint keep y  0x08048080 lab09-2.asm:12
      breakpoint already hit 1 time
(gdb) break *0x80480b1
Breakpoint 2 at 0x80480b1: file lab09-2.asm, line 25.
(gdb) i b
Num   Type      Disp Enb Address  What
1     breakpoint keep y  0x08048080 lab09-2.asm:12
      breakpoint already hit 1 time
2     breakpoint keep y  0x080480b1 lab09-2.asm:25
(gdb) si
(gdb) si
(gdb) |
```

Рис. 13. Выполнение 2 инструкции с помощью si

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

--Register group: general--
eax      0x4      4
ecx      0x8049000 134516736
edx      0x0      0
ebx      0x1      1
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x804808f 0x804808f <_start+15>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+ 0x8048080 <_start>      mov     eax,0x4
0x8048085 <_start+5>      mov     ebx,0x1
0x804808a <_start+10>     mov     ecx,0x8049000
>0x804808f <_start+15>     mov     edx,0x8
0x8048094 <_start+20>     int     0x80
0x8048096 <_start+22>     mov     eax,0x4
0x804809b <_start+27>     mov     ebx,0x1
0x80480a0 <_start+32>     mov     ecx,0x8049000
0x80480a5 <_start+37>     mov     edx,0x7
0x80480aa <_start+42>     int     0x80
0x80480ac <_start+44>     mov     eax,0x1
b+ 0x80480b1 <_start+49>     mov     ebx,0x0
0x80480b6 <_start+54>     int     0x80

native process 8855 (asm) In: _start L15 PC: 0x804808f
(gdb) info breakpoints
Num    Type      Disp Enb Address  What
1      breakpoint keep y  0x8048080 lab09-2.asm:12
      breakpoint already hit 1 time
(gdb) break *0x80480b1
Breakpoint 2 at 0x80480b1: file lab09-2.asm, line 25.
(gdb) i b
Num    Type      Disp Enb Address  What
1      breakpoint keep y  0x8048080 lab09-2.asm:12
      breakpoint already hit 1 time
2      breakpoint keep y  0x80480b1 lab09-2.asm:25
(gdb) si
(gdb) si
(gdb) si
(gdb) si
```

Рис. 14. Выполнение 3 инструкции с помощью si

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

--Register group: general--
eax      0x4      4
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048094 0x8048094 <_start+20>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+ 0x8048080 <_start>      mov     eax,0x4
0x8048085 <_start+5>      mov     ebx,0x1
0x804808a <_start+10>     mov     ecx,0x8049000
0x804808f <_start+15>     mov     edx,0x8
>0x8048094 <_start+20>     int     0x80
0x8048096 <_start+22>     mov     eax,0x4
0x804809b <_start+27>     mov     ebx,0x1
0x80480a0 <_start+32>     mov     ecx,0x8049000
0x80480a5 <_start+37>     mov     edx,0x7
0x80480aa <_start+42>     int     0x80
0x80480ac <_start+44>     mov     eax,0x1
b+ 0x80480b1 <_start+49>     mov     ebx,0x0
0x80480b6 <_start+54>     int     0x80

native process 8855 (asm) In: _start L16 PC: 0x8048094
Num    Type      Disp Enb Address  What
1      breakpoint keep y  0x8048080 lab09-2.asm:12
      breakpoint already hit 1 time
(gdb) break *0x80480b1
Breakpoint 2 at 0x80480b1: file lab09-2.asm, line 25.
(gdb) i b
Num    Type      Disp Enb Address  What
1      breakpoint keep y  0x8048080 lab09-2.asm:12
      breakpoint already hit 1 time
2      breakpoint keep y  0x80480b1 lab09-2.asm:25
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
```

Рис. 15. Выполнение 4 инструкции с помощью si

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

Register group: general
eax      0x8      8
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048096 0x8048096 <_start+22>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+ 0x8048080 <_start>    mov    eax,0x4
0x8048085 <_start+5>    mov    ebx,0x1
0x804808a <_start+10>   mov    ecx,0x8049000
0x804808f <_start+15>   mov    edx,0x8
0x8048094 <_start+20>   int    0x80
>0x8048096 <_start+22>  mov    eax,0x4
0x804809b <_start+27>  mov    ebx,0x1
0x80480a0 <_start+32>  mov    ecx,0x8049008
0x80480a5 <_start+37>  mov    edx,0x7
0x80480aa <_start+42>  int    0x80
0x80480ac <_start+44>  mov    eax,0x1
b+ 0x80480b1 <_start+49>  mov    ebx,0x0
0x80480b6 <_start+54>  int    0x80

native process 8855 (asm) In: _start L18 PC: 0x8048096
1 breakpoint keep y 0x8048080 lab09-2.asm:12
  breakpoint already hit 1 time
(gdb) break *0x80480b1
Breakpoint 2 at 0x80480b1: file lab09-2.asm, line 25.
(gdb) i b
Num    Type             Disp Enb Address      What
1      breakpoint        keep y 0x8048080 lab09-2.asm:12
      breakpoint already hit 1 time
2      breakpoint        keep y 0x80480b1 lab09-2.asm:25
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) siHello,
(gdb) |
```

Рис. 16. Выполнение 5 инструкции с помощью si

Просматриваем содержимое регистров с помощью команды info registers

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

Register group: general
eax      0x8      8
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048096 0x8048096 <_start+22>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+ 0x8048080 <_start>    mov    eax,0x4
0x8048085 <_start+5>    mov    ebx,0x1
0x804808a <_start+10>   mov    ecx,0x8049000
0x804808f <_start+15>   mov    edx,0x8
0x8048094 <_start+20>   int    0x80
>0x8048096 <_start+22>  mov    eax,0x4
0x804809b <_start+27>  mov    ebx,0x1
0x80480a0 <_start+32>  mov    ecx,0x8049008
0x80480a5 <_start+37>  mov    edx,0x7
0x80480aa <_start+42>  int    0x80
0x80480ac <_start+44>  mov    eax,0x1
b+ 0x80480b1 <_start+49>  mov    ebx,0x0
0x80480b6 <_start+54>  int    0x80

native process 8855 (asm) In: _start L18 PC: 0x8048096
eax      0x8      8
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048096 0x8048096 <_start+22>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       Hello, 0x2b 43
--Type <RET> for more, q to quit, c to continue without paging--
```

Рис. 17. Команда info registers

С помощью команды `x &<имя переменной>` просматриваем значение переменной `msg1` по имени и значение `msg2` по адресу. Далее с помощью команды `set` изменяем первый символ переменной `msg1` на `'h'` и первый символ переменной `msg2` на `'W'`.

```

iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B+>0x8048080 <_start> mov    eax,0x4
0x8048085 <_start+5> mov    ebx,0x1
0x804808a <_start+10> mov    ecx,0x8049000
0x804808f <_start+15> mov    edx,0x8
0x8048094 <_start+20> int    0x80
0x8048096 <_start+22> mov    eax,0x4
0x804809b <_start+27> mov    ebx,0x1
0x80480a0 <_start+32> mov    ecx,0x8049008
0x80480a5 <_start+37> mov    edx,0x7
0x80480aa <_start+42> int    0x80
0x80480ac <_start+44> mov    eax,0x1
b+ 0x80480b1 <_start+49> mov    ebx,0x0
0x80480b6 <_start+54> int    0x80

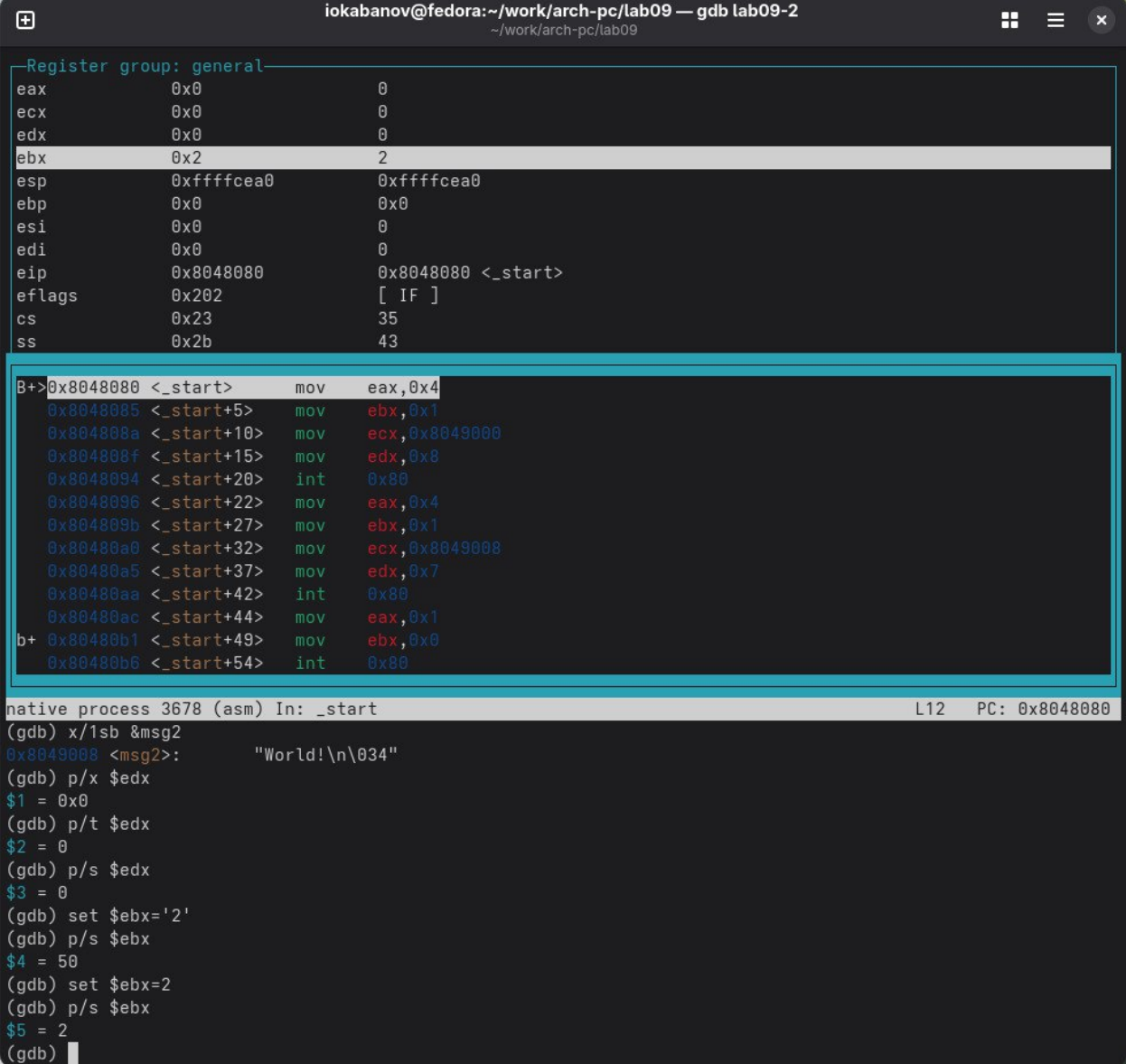
native process 3678 (asm) In: _start L12 PC: 0x8048080
es      0x2b     43
--Type <RET> for more, q to quit, c to continue without paging--
fs      0x0      0
gs      0x0      0
(gdb) x/1sb &msg1
0x8049000 <msg1>: "Hello, "
(gdb) x/1sb 0x8049008
0x8049008 <msg2>: "world!\n\034"
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x8049000 <msg1>: "hello, "
(gdb) set {char}&msg2='W'
(gdb) x/1sb &msg2
0x8049008 <msg2>: "World!\n\034"
(gdb)

```

Рис. 18. Меняем значения переменных `msg1` и `msg2`

С помощью `r/F` выводим в различных форматах (в шестнадцатеричном формате - `x`, в двоичном формате - `t` и в символьном виде - `s`) значение регистра `edx`. С помощью команды `set` меняем значение регистра `ebx` сначала на `'2'`, а затем на `2` и выводим с помощью `r/s`. Значение `'2'` интерпретируется как символ и для него в

регистр ebx записывается ASCII код числа 2 который и выводится командой p/s. Значение 2 записывается как целое число в ebx и поэтому p/s тоже выводит 2.



The screenshot shows a GDB terminal window titled "iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2". The window is divided into two main sections. The top section, titled "Register group: general", displays the values of various registers: eax (0x0), ecx (0x0), edx (0x0), ebx (0x2), esp (0xffffcea0), ebp (0x0), esi (0x0), edi (0x0), eip (0x8048080), eflags (0x202), cs (0x23), and ss (0x2b). The bottom section shows assembly code starting at address 0x8048080. The code includes instructions like "mov eax, 0x4", "mov ebx, 0x1", "mov ecx, 0x8049000", "mov edx, 0x8", "int 0x80", and "mov ebx, 0x0". The GDB prompt "(gdb)" is visible at the bottom.

```
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x2      2
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43

B> 0x8048080 <_start>   mov     eax, 0x4
    0x8048085 <_start+5> mov     ebx, 0x1
    0x804808a <_start+10> mov     ecx, 0x8049000
    0x804808f <_start+15> mov     edx, 0x8
    0x8048094 <_start+20> int     0x80
    0x8048096 <_start+22> mov     eax, 0x4
    0x804809b <_start+27> mov     ebx, 0x1
    0x80480a0 <_start+32> mov     ecx, 0x8049000
    0x80480a5 <_start+37> mov     edx, 0x7
    0x80480aa <_start+42> int     0x80
    0x80480ac <_start+44> mov     eax, 0x1
b+  0x80480b1 <_start+49> mov     ebx, 0x0
    0x80480b6 <_start+54> int     0x80

native process 3678 (asm) In: _start L12 PC: 0x8048080
(gdb) x/1sb &msg2
0x8049008 <msg2>: "World!\n\034"
(gdb) p/x $edx
$1 = 0x0
(gdb) p/t $edx
$2 = 0
(gdb) p/s $edx
$3 = 0
(gdb) set $ebx='2'
(gdb) p/s $ebx
$4 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$5 = 2
(gdb)
```

Рис. 19. Использование команды p/F

Завершаем выполнение программы с помощью команды continue и выходим из GDB с помощью команды quit.

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb lab09-2
~/work/arch-pc/lab09

eax      0x0      0
eax      0x1      1
ecx      0x8049008 134516744
edx      0x7      7
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eip      0x80480b1 0x80480b1 <_start+49>
cs       0x23     35
ss       0x2b     43

B> 0x8048080 <_start> mov    eax,0x4
    0x804809b <_start+27> mov    ebx,0x1
    0x80480a0 <_start+32> mov    ecx,0x8049008
    0x80480a5 <_start+37> mov    edx,0x7
    0x80480aa <_start+42> int    0x80
    0x80480ac <_start+44> mov    eax,0x1
b+ 0x80480b1 <_start+49> mov    ebx,0x0
B> 0x80480b1 <_start+49> mov    ebx,0x0

    0x80480b8 add    BYTE PTR [eax],al
    0x80480ba add    BYTE PTR [eax],al
    0x80480bc add    BYTE PTR [eax],al
    0x80480be add    BYTE PTR [eax],al
    0x80480c0 add    BYTE PTR [eax],al

native process 3678 (asm) In: _start L12 PC: 0x8048080
(gdb) x/1sb &msg2 25 0b1
$2 = 0
(gdb) p/s $edx
$3 = 0
(gdb) set $ebx='2'
(gdb) p/s $ebx
$4 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$5 = 2
(gdb) continuehello, World!

Continuing.

Breakpoint 2, _start () at lab09-2.asm:25
(gdb) quit
```

Рис. 20. Выход из GDB

### 9.4.2.3. Обработка аргументов командной строки в GDB

Копируем файл lab8-2.asm, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки (Листинг 8.2) в файл с именем lab09-3.asm и создаем исполняемый файл. Загружаем исполняемый файл в отладчик, указав аргументы. Устанавливаем точку останова перед первой инструкцией(b \_start) в программе и запускаем ее.



```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb --args lab09-3 аргумент1 аргумент 2 аргумент 3
~/work/arch-pc/lab09

iokabanov@fedora:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
iokabanov@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
iokabanov@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
iokabanov@fedora:~/work/arch-pc/lab09$ gdb --args lab09-3 аргумент1 аргумент 2 'аргумент 3'
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) b _start
Breakpoint 1 at 0x8048148: file lab09-3.asm, line 11.
(gdb) run
Starting program: /home/iokabanov/work/arch-pc/lab09/lab09-3 аргумент1 аргумент 2 аргумент\ 3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Breakpoint 1, _start () at lab09-3.asm:11
11      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) x/x $esp
0xffffce50: 0x00000005
```

Рис. 21. Запускаем lab09-3

Смотрим на остальные позиции стека. Шаг изменения адреса равен 4, потому что программа скомпилирована и выполняется в 32-битном режиме. Поскольку в 1 байте 8 бит, размер адреса в байтах составляет:

$$32 \text{ бита} / 8 = 4 \text{ байта}$$

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb --args lab09-3 аргумент1 аргумент2 аргумент3
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) b _start
Breakpoint 1 at 0x8048148: file lab09-3.asm, line 11.
(gdb) run
Starting program: /home/iokabanov/work/arch-pc/lab09/lab09-3 аргумент1 аргумент2 аргумент\ 3

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Breakpoint 1, _start () at lab09-3.asm:11
11      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) x/x $esp
0xffffce50: 0x00000005
(gdb) x/s *(void**)(esp + 4)
0xffffd039: "/home/iokabanov/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)(esp + 8)
0xffffd064: "аргумент1"
(gdb) x/s *(void**)(esp + 12)
0xffffd076: "аргумент"
(gdb) x/s *(void**)(esp + 16)
0xffffd087: "2"
(gdb) x/s *(void**)(esp + 20)
0xffffd089: "аргумент 3"
(gdb) x/s *(void**)(esp + 24)
0x0: <error: Cannot access memory at address 0x0>
(gdb) █
```

Рис. 22. Просмотр позиций стека с шагом 4

## 9.5. Задание для самостоятельной работы

Задание 1: Преобразуйте программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции  $f(x)$  как подпрограмму.

Преобразовываем программу из лабораторной работы №8, добавив подпрограмму `_calc_f`. Создаем исполняемый файл и убеждаемся в корректной работе программы.

Листинг измененной программы (файл `task1.asm`):

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
msg db "Результат: ",0
```

```
SECTION .text
```

```
global _start
```

```
_start:
```

```
    pop ecx
```

```
    pop edx
```

```
    sub ecx, 1
```

```
    mov esi, 0
```

```
next:
```

```
    cmp ecx, 0h
```

```
    jz _end
```

```
    pop eax
```

```
    call atoi
```

```
    call _calc_f
```

```
    add esi, eax
```

```
    loop next
```

```
_end:
```

```
    mov eax, msg
```

```
    call sprint
```

```
    mov eax, esi
```

```
    call iprintLF
```

```
    call quit
```

```
; -----
```

```
; Подпрограмма вычисления функции  $f(x) = 30x - 11$ 
```

```

; -----
_calc_f:
    imul eax, 30 ; Умножаем x на 30
    sub eax, 11 ; Вычитаем 11
    ret         ; Возврат управления в основную программу

```

```

task1.asm [----] 58 L: [ 1+40 41/ 42] *(825 / 826b) 0010 0x00A [*][X]
#include 'in_out.asm'

SECTION .data
msg db "Результат: ",0

SECTION .text
global _start

_start:
    pop ecx.....
    pop edx.....
    sub ecx, 1...
    ....
    mov esi, 0...

next:
    cmp ecx, 0h..
    jz _end.....
    ....
    pop eax.....
    call atoi....
    ....
    call _calc_f.
    add esi, eax.
    loop next....

_end:
    mov eax, msg.
    call sprint
    mov eax, esi.
    call iprintLF
    ....
    call quit

; -----
; Подпрограмма вычисления функции f(x) = 30x - 11
; -----
_calc_f:
    imul eax, 30 ; Умножаем x на 30
    sub eax, 11 ; Вычитаем 11
    ret         ; Возврат управления в основную программу

```

1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пере-чить 7Поиск 8Удалить 9МенюМС 10Выход

Рис. 23. Измененная программа из лабораторной работы №8

```

iokabanov@fedora:~/work/arch-pc/lab09
iokabanov@fedora:~/work/arch-pc/lab09$ nasm -f elf task1.asm
iokabanov@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o task1 task1.o
iokabanov@fedora:~/work/arch-pc/lab09$ ./task1 6
Результат: 169
iokabanov@fedora:~/work/arch-pc/lab09$

```

Рис. 24. Запуск task1

Задание 2: В листинге 9.3 приведена программа вычисления выражения  $(3 + 2) * 4 + 5$ . При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

Создаем файл task2.asm для программы из листинга 9.3. Создаем исполняемый файл и проверяем, что программа действительно дает неверный результат (10 вместо 25). С помощью отладчика GDB анализируем изменение значений регистров, определяем ошибку (рассогласование использования EAX и EBX) и исправляем ее. Запускаем новый исполняемый файл и убеждаемся в корректной работе программы.

Листинг task2.asm (после изменения):

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
div: DB 'Результат: ',0
```

```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
; ---- Вычисление выражения (3+2)*4+5
```

```
    mov eax,3    ; загружаем первое число в EAX (для будущей работы с mul)
```

```
    mov ebx,2    ; второе число в EBX
```

```
    add eax,ebx
```

```
    mov ecx,4
```

```
    mul ecx
```

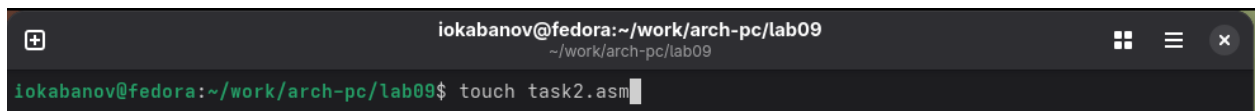
```
    add eax,5    ; прибавляем 5 к EAX. EAX = 20 + 5 = 25
```

```
    mov edi,eax  ; сохраняем результат в EDI для вывода
```

```
; ---- Вывод результата на экран
```

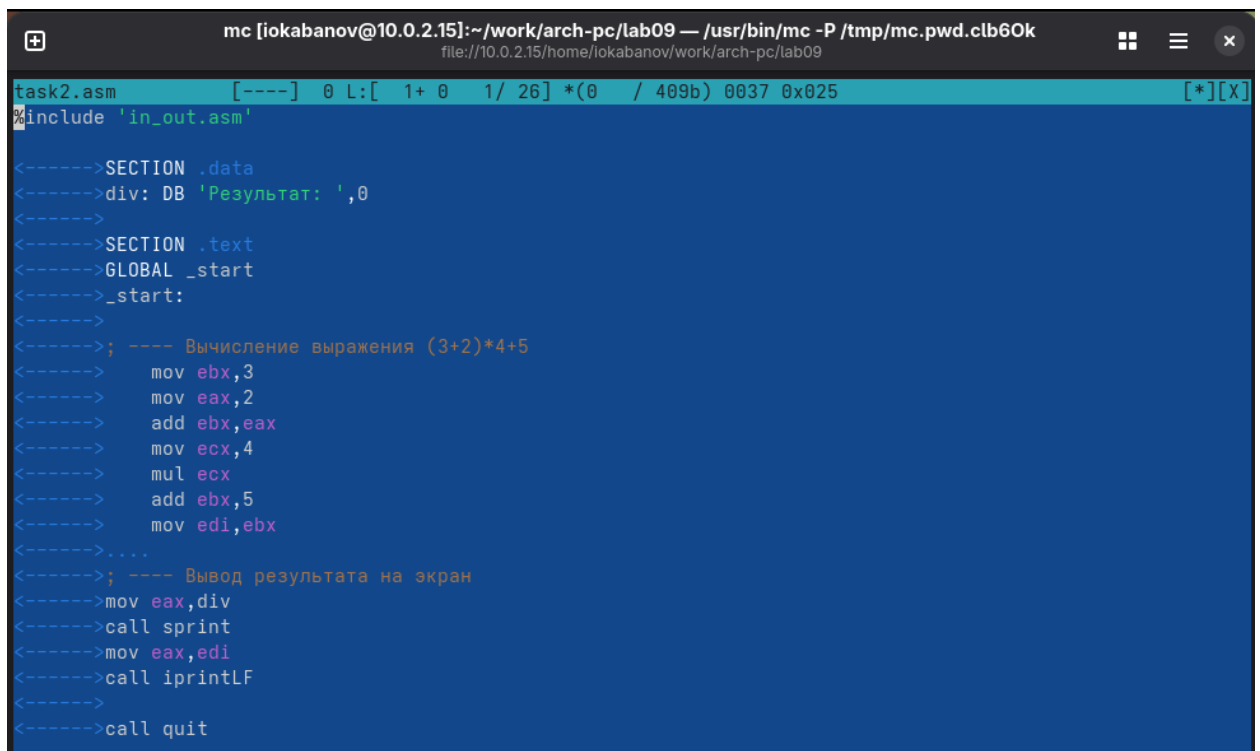
```
mov eax,div
call sprint
mov eax,edi
call iprintLF

call quit
```



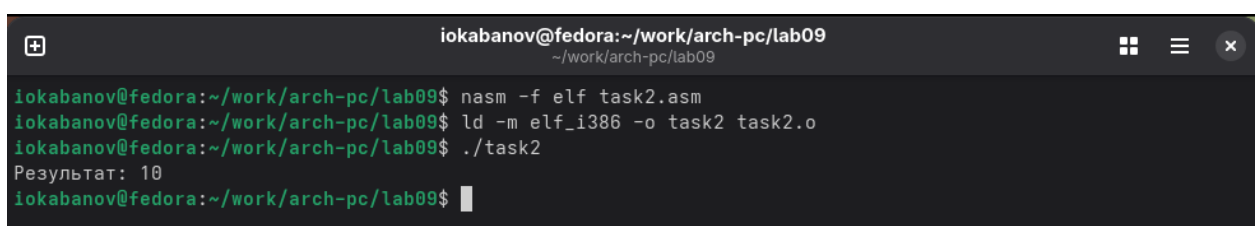
A terminal window titled 'iokabanov@fedora:~/work/arch-pc/lab09' with a dark background. The command 'touch task2.asm' has been executed, and the prompt is now 'iokabanov@fedora:~/work/arch-pc/lab09\$'.

Рис. 25. Создаем task2.asm



An MC editor window titled 'mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab09' with a dark blue background. The file 'task2.asm' is open, showing assembly code with comments in Russian. The code includes a data section for a string, a text section with a global start label, and instructions to calculate (3+2)\*4+5 and print the result. Comments are separated from code by '<----->'. The code ends with a 'call quit' instruction.

Рис. 26. Программа из листинга 9.3



A terminal window titled 'iokabanov@fedora:~/work/arch-pc/lab09' with a dark background. The following commands and output are shown: 'nasm -f elf task2.asm', 'ld -m elf\_i386 -o task2 task2.o', and './task2'. The output of the program is 'Результат: 10'.

Рис. 27. Неверный вывод программы из листинга 9.3

```
iokabanov@fedora:~/work/arch-pc/lab09 — gdb task2
~/work/arch-pc/lab09

Register group: general
eax      0x2      2
ecx      0x4      4
edx      0x0      0
ebx      0x5      5
esp      0xffffcea0 0xffffcea0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048179 0x8048179 <_start+17>
eflags   0x206    [ PF IF ]
cs       0x23     35
ss       0x2b     43

B+ 0x8048168 <_start>    mov     ebx,0x3
0x804816d <_start+5>    mov     eax,0x2
0x8048172 <_start+10>   add     ebx,eax
0x8048174 <_start+12>   mov     ecx,0x4
>0x8048179 <_start+17>  mul     ecx
0x804817b <_start+19>   add     ebx,0x5
0x804817e <_start+22>   mov     edi,ebx
0x8048180 <_start+24>   mov     eax,0x8049000
0x8048185 <_start+29>   call    0x804808f <sprint>
0x804818a <_start+34>   mov     eax,edi
0x804818c <_start+36>   call    0x8048106 <iprintLF>
0x8048191 <_start+41>   call    0x804815b <quit>
0x8048196             add     BYTE PTR [eax],al

native process 12054 (asm) In: _start L?? PC: 0x8048179
(gdb) layout regs
(gdb) si
0x0804816d in _start ()
(gdb) si
0x08048172 in _start ()
(gdb) si
0x08048174 in _start ()
(gdb) si
0x08048179 in _start ()
(gdb) |
```

Рис. 28. С помощью GDB анализируем изменения значений регистров

```
mc [iokabanov@10.0.2.15]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.pwd.GRz0zc
file://10.0.2.15/home/iokabanov/work/arch-pc/lab09

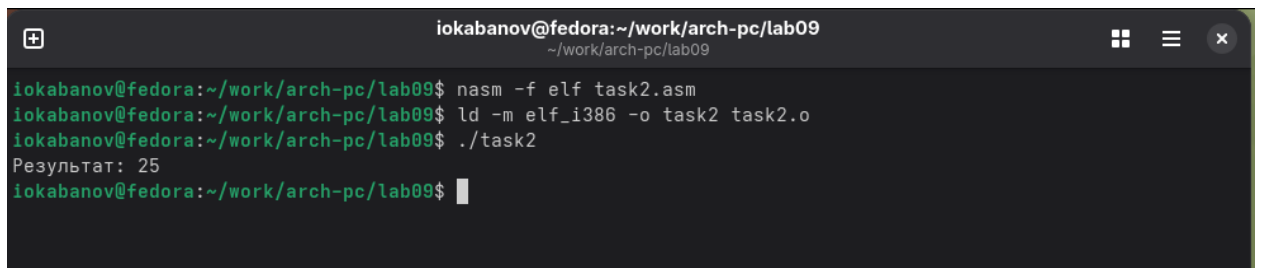
task2.asm [----] 0 L: [ 1+25 26/ 26 ] *(650 / 650b) <EOF> [*][X]
#include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0

SECTION .text
GLOBAL _start
_start:

; ---- Вычисление выражения (3+2)*4+5
mov eax,3 ; загружаем первое число в EAX (для будущей работы с mul)
mov ebx,2 ; второе число в EBX
add eax,ebx
mov ecx,4
mul ecx
add eax,5 ; прибавляем 5 к EAX. EAX = 20 + 5 = 25
mov edi,eax ; сохраняем результат в EDI для вывода
....
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Рис. 29. Меняем программу task2.asm

A terminal window with a dark background. The title bar shows the user 'iokabanov@fedora' and the path '~/work/arch-pc/lab09'. The terminal contains the following text:

```
iokabanov@fedora:~/work/arch-pc/lab09$ nasm -f elf task2.asm
iokabanov@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o task2 task2.o
iokabanov@fedora:~/work/arch-pc/lab09$ ./task2
Результат: 25
iokabanov@fedora:~/work/arch-pc/lab09$
```

Рис. 30. Запускаем измененную программу task2 и убеждаемся в корректной работе

## Вывод

В ходе выполнения лабораторной работы были приобретены навыки написания программ на NASM с использованием подпрограмм. Были получены знания об отладке, работе с методами отладки при помощи GDB и его основными возможностями.